OPERATING GUIDE / 20 OCTOBER 2026

How the grader works now

An illustrated guide to submissions, updates, evaluation runs, SLURM jobs and recovery.

The central idea
The coordinator decides what to grade. SLURM executes it. The Cloudflare Worker displays grading status and published results. Local grading state connects these three parts.

This guide describes the current grader code in this workspace, including the recent durable-job and Worker-publication changes. It is written to explain the behavior step by step, rather than assume familiarity with the new classes.

Start with the distinctions

Question	Setting / concept
Which repositories should be checked?	Discovery: **submissions** or **updates**.
How are students grouped into compute jobs?	Batching: **per_task** or **per_submission**.
Does this coordinator stay running?	Execution: **wait_for_jobs** true or false.
Where are results published?	**worker_publication**, plus the independent optional GitHub pusher.

Reading map

Pages	What you will learn
2-4	Actors, vocabulary and independent configuration choices.
5-9	Both discovery modes and a complete updates-mode example.
10-15	Batching, persistent state, passes and scheduler recovery.
16-19	Publication, acknowledgement, compute execution and regrading.
20-22	Configuration, troubleshooting, code map and quick answers.

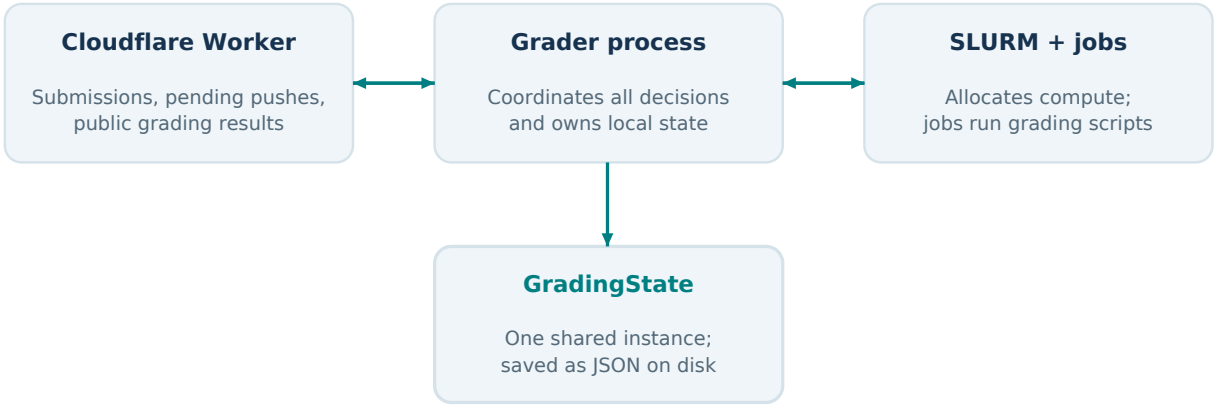
Examples use short labels such as S-A, R-A-CPU, T1 and J1. These are explanatory aliases: real submission/run IDs are UUIDs, submission tokens are UUID hex strings, and real commit hashes are full Git hashes.

Scope and deployment
Worker-enabled examples assume the matching run-based Worker API is deployed. This guide does not assert that the remote Worker or Leonardo deployment has already been upgraded.

HOW THE GRADER WORKS

Who does what?

The Worker is a web service. A SLURM job is a compute execution. They are different things.



Inside the grader: WorkerPublication talks left; DurableJobs + SlurmRunner talk right.

Component	Responsibility
Cloudflare Worker	Returns submissions and pending pushes; stores remote run/publication state; serves the public leaderboard.
Grader	Coordinates passes, pins inputs, decides eligibility, ingests results, finishes snapshots and exports the local leaderboard.
DurableJobs	Creates evaluation records, saves scheduler submission intents, adopts job IDs and reconciles uncertain submissions.
SlurmRunner	Uses submitit to submit jobs and scheduler commands to discover their state. It is the scheduler adapter.
SubmissionGrader	Runs inside a compute job: clone the bundle, execute grading, write result files and clean up its clone.
WorkerPublication	Starts remote runs and sends durable completion, cancellation and acknowledgement requests.
GradingState	Shared local state object, persisted in grading_state.json. All coordinator helpers use the same instance.

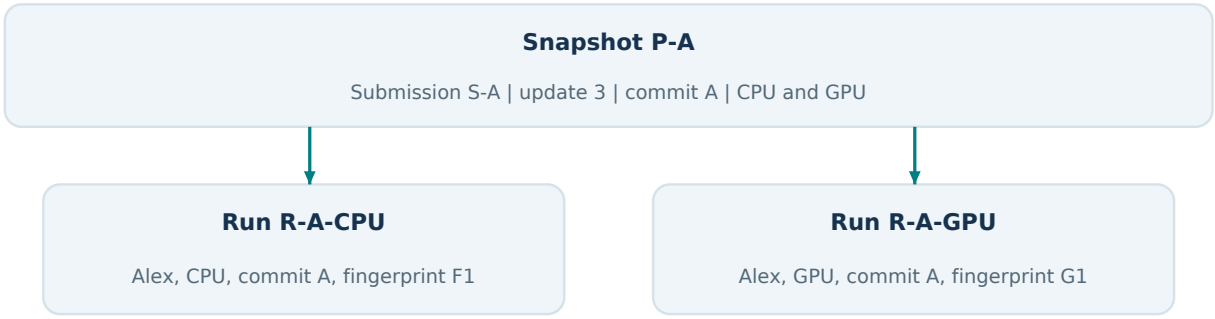
There is no chain that every state access must follow
Grader, DurableJobs and WorkerPublication all access the same GradingState instance. They are in-process helpers. Creating a DurableJobs object does not create a job.

HOW THE GRADER WORKS

The identities, precisely

A push, a snapshot, an evaluation and a scheduler submission have different lifetimes.

Identity	Meaning	Example
Submission ID	Worker identity for the student submission/repository within an assignment.	S-A = Alex
update_id	Integer push version scoped to that submission. Nullable when grading without known push provenance.	3
Snapshot ID	Local record of a pinned repository/commit, applicable tasks and their fingerprints.	P-A
run_id	One student/task evaluation of fixed inputs. Retained across replacement scheduler attempts.	R-A-CPU
submission_token	One local intent to submit one batch. Saved before contacting SLURM; used in the job name.	T1
SLURM job ID	The allocation SLURM actually accepted. Learned from submission or recovered by token lookup.	J1
perf_hash	Fingerprint of the task script/configuration. Detects when the same commit needs a fresh evaluation.	F1
revision	Worker-assigned order of accepted runs for one submission/task; not a job counter.	1, 2, 3...



Update = push version. Run = evaluation. Token = submission attempt. Job ID = SLURM allocation.

Two relationships matter: one scheduler token can contain several run IDs; one run ID can appear in several scheduler tokens over retries. A scheduler retry keeps the evaluation identity. A genuinely new evaluation gets a new run ID.

HOW THE GRADER WORKS

Four independent choices

In this guide, "the other discovery mode" means submissions mode. It does not mean a different batching strategy.

Setting	Choices	What changes
worker_discovery	submissions / updates	Which repositories are selected for HEAD checks and snapshot creation.
job_mode	per_task / per_submission	Whether one task job contains several students or a single student.
wait_for_jobs	true / false	Repeat local passes or exit after one pass.
worker_publication	true / false	Send run state/results to the Worker or only retain local output and optional GitHub publishing.

Example A: updates discovery with batched jobs

```
grader:
  worker_publication: true
  worker_discovery: "updates"
  job_mode: "per_task"
  wait_for_jobs: false
```

Use pending pushes to reduce repository checks. Grade several students in each task job. Make one coordinator pass, then let a later scheduled invocation continue.

Example B: submissions discovery with the same publication

```
grader:
  worker_publication: true
  worker_discovery: "submissions"
  job_mode: "per_submission"
  wait_for_jobs: true
```

Check every listed submission. Give each student/task its own job. Keep making passes while scheduler work or job-limit deferrals remain.

Publication is a separate switch
Both discovery modes can publish to the Worker. Updates discovery requires worker_publication: true. Submissions discovery also works with publication disabled; that remains the default configuration.

HOW THE GRADER WORKS

Submissions mode

Check the complete assignment list, then use local records to avoid unnecessary grading.

```
worker_publication: true
worker_discovery: "submissions"
```

At the first snapshot lookup for an assignment in a coordinator invocation, the grader calls **POST /submissions**. It receives the assignment's submission records, including repository URLs and deploy keys. It does not require a pending webhook notification.

For each selected repository with applicable tasks, it either resumes a compatible unfinished local pin or resolves actual Git HEAD. A task needs grading if the local result is missing, or its repository URL, commit or performance fingerprint differs.

Student	What the grader finds	Outcome
Alex	Same repository, commit A and CPU fingerprint F1 as the saved result.	No new CPU compute.
Bianca	Saved result is at B; HEAD is now C.	Create an evaluation for C.
Carlo	Same commit D, but the CPU test changed from F1 to F2.	Create an evaluation using F2.
Daria	No result for the newly configured GPU task.	Grade GPU, even without a push notification.

Publication still follows the same lifecycle

For work that needs compute: start the remote run, submit SLURM work, ingest the result and publish it. Existing grades remain visible during replacement grading. Already completed local results can also be imported into Worker runs without repeating compute.

No update version is required
If /submissions provides no update_id, the evaluation uses null push provenance. Results can still be started and published. No push acknowledgement is queued for a snapshot whose update_id is null.

The assignment list and successful commit lookups are cached for the current coordinator invocation. Repeated waiting passes do not continuously reload the complete list or move a successful pin to a newer HEAD.

HOW THE GRADER WORKS

Updates mode

Pending pushes narrow ordinary repository checks; local state and full reconciliation still matter.

```
worker_publication: true
worker_discovery: "updates"
worker_reconcile_seconds: 3600
```

The current implementation first reads **POST /submissions** for assignment metadata. It also reads **GET /updates**, then selects entries to process. Updates mode primarily saves Git HEAD lookups; it does not eliminate the submissions API call.

A submission is selected if any of these is true

- A full scan is due: first use, task fingerprints changed, or the configured reconciliation interval elapsed.
- The Worker reports a pending update for that submission.
- Local state contains an unfinished snapshot for it.
- The student has no local assignment records yet.

Ordinary scan, full scan not due	Selected ?	Why
Alex: pending update 3	Yes	Resolve/resume the snapshot for that work.
Bianca: no pending update, no unfinished work, already recorded	No	Avoid another HEAD lookup this invocation.
Carlo: no pending update, unfinished local snapshot	Yes	Recover old work, using its saved commit.
Daria: no pending update, no local records	Yes	Initial grading must not depend on a webhook.

The first updates-mode invocation can look like submissions mode
The first scan is a full scan. Adding a task or changing its fingerprint also forces a full scan. An empty /updates response does not imply there is no local work to resume or publish.

Discovery occurs once per assignment per coordinator invocation. New pushes and an elapsed full-scan interval are evaluated when a fresh invocation performs discovery, not by re-fetching updates on every 30-second waiting pass.

HOW THE GRADER WORKS

Updates example: discover and pin

We follow Alex through both tasks. Assume no previous jobs or results.

Assignment: vectorsum. **Student:** Alex, submission S-A. **Tasks:** CPU and GPU. **Configuration:** publication enabled, updates discovery, CPU blocking. The Worker reports update 3.

```
GET /updates

{
  "submissions": [{
    "id": "S-A",
    "assignment_name": "vectorsum",
    "student_id": 1001,
    "repo_ssh_url": "git@github.com:alex/vectorsum.git",
    "private_key": "<secret deploy key, omitted>",
    "update_id": 3,
    "last_commit": "<webhook hint>"
  }]
}
```

This is an abbreviated teaching response: IDs and commits are labels, and other metadata fields are omitted. The real response contains full identities and sensitive credentials.

The grader now fixes the actual input

- Select S-A from the metadata/update lists. Reading /updates does not consume update 3.
- Resolve Git HEAD using the deploy key. Suppose the actual commit is A. The webhook hint does not override this lookup.
- Save local snapshot P-A: repository URL, commit A, update 3, tasks CPU/GPU and fingerprints F1/G1.
- Download commit A into a shared Git bundle before submitting compute work. A compatible unfinished pin would be reused instead of choosing another HEAD.

An update ID is not a commit identity
Update 3 records which notification snapshot the coordinator is processing. If HEAD advanced before lookup, its actual pinned commit may differ from the webhook hint. The actual graded commit is always recorded explicitly.

HOW THE GRADER WORKS

Example: start a run, then a job

The evaluation is registered remotely before compute is submitted.

1. Create the CPU evaluation R-A-CPU

DurableJobs.ensure_run() creates a local run for Alex/CPU at commit A and fingerprint F1. Creation and its snapshot membership are saved together. The run stores update_id 3. No SLURM allocation exists yet.

2. Register that evaluation with the Worker

WorkerPublication inspects the latest remote run for this submission/task, saves an immutable start payload, then sends it. With no predecessor, previous_run_id is null.

```
POST /updates/grading  (IDs/hashes abbreviated)
{
  "id": "S-A", "task": "CPU", "run_id": "R-A-CPU",
  "previous_run_id": null, "update_id": 3,
  "name": "alex", "commit_hash": "A", "perf_hash": "F1"
}
```

The Worker confirms the run is active. The leaderboard may now show CPU as grading, even if its SLURM job has not started. Any previous CPU result remains visible. If this start cannot be confirmed, this work is not submitted to SLURM.

3. Submit the scheduler attempt T1

DurableJobs saves an intent with token T1 and run membership [R-A-CPU]. It first saves prepared, then submitting immediately before the runner call. SlurmRunner submits with a real name of the form gw-<32 hex digits>. When SLURM returns J1, the intent is adopted and saved as accepted.

```
Snapshot P-A  ->  update 3, commit A, CPU + GPU
Run R-A-CPU  ->  Alex / CPU / A / F1
Token T1    ->  submission attempt containing R-A-CPU
Job J1      ->  SLURM allocation discovered for T1
```

Because CPU is blocking in this example, GPU waits until the CPU task has no jobs, holds, deferrals or submission failures that block later tasks. Blocking applies across this assignment, not just Alex.

HOW THE GRADER WORKS

Updates example: publish and ack

Compute completion, remote publication and push acknowledgement are three separate milestones.

Moment	Coordinator / compute action	What the Worker can show
CPU job runs	J1 clones the bundle and executes the CPU grading script.	CPU grading is active.
CPU result exists	Job writes an atomic result file. Coordinator reads it on a pass.	Previous result may still be visible.
CPU saved locally	Validate identity and result shape; save result plus completion outbox; then delete file.	Remote update may still be pending.
CPU published	POST /updates/graded is confirmed for R-A-CPU.	New CPU result; CPU no longer grading.
GPU evaluation	Create/start R-A-GPU; submit its own scheduler attempt; save and publish its result.	GPU may show grading while CPU is complete.
Snapshot ready	Both task runs have confirmed completed publication. Persist acknowledgement evidence.	Both new results are visible.
Push acknowledged	POST /updates/acknowledged for update 3 cites both completed run IDs.	Matching pending push is removed, if not superseded.

```
POST /updates/acknowledged  (abbreviated)
{
  "id": "S-A", "update_id": 3,
  "runs": {"CPU": "R-A-CPU", "GPU": "R-A-GPU"}
}
```

Finishing CPU does not consume the whole update
The snapshot is not ready until every applicable task has completed published evidence. Conversely, a result can be published while its batch job is still doing other students. Acknowledgement concerns the declared evaluations, not whether every batch process has exited.

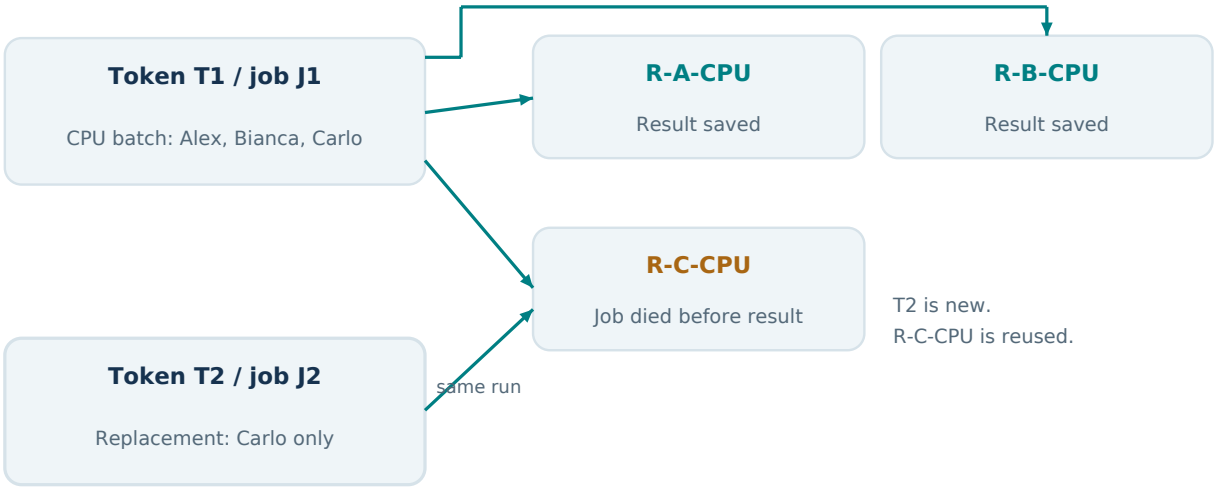
If the coordinator exits after CPU submission, nothing about the workflow changes: the next invocation resumes from the saved snapshot, run and scheduler intent.

HOW THE GRADER WORKS

One job can contain several runs

Batching changes compute grouping. It does not merge student evaluations.

Suppose Alex, Bianca and Carlo all need CPU grading at their own pinned commits. Each has a separate evaluation: R-A-CPU, R-B-CPU and R-C-CPU.



With job_mode: per_task

The coordinator submits one CPU batch containing all three eligible submissions. A single SubmissionGrader processes them sequentially and writes each result as soon as it is ready. If J1 dies during Carlo, Alex and Bianca keep their completed results. A later batch contains only Carlo's missing work.

With job_mode: per_submission

The coordinator submits three separate scheduler attempts, one per student/CPU evaluation, subject to available slots. Each token normally points to one run ID. This gives independent jobs but consumes more scheduler slots.

What is retried?	Run ID	Token / job ID
Carlo's evaluation after a failed attempt	Keep R-C-CPU.	Create T2 and obtain J2.
Carlo at a new commit or task fingerprint	Create a new run.	Create a new token and job.
An uncertain HTTP completion request	Keep the same run.	No compute submission; retry the saved API payload.

No durable object per job
DurableJobs is the scheduler-management helper. The durable things are records inside GradingState: evaluation runs and submission intents. The coordinator creates short-lived helper objects repeatedly around the same state instance.

Source: grader/grader.py: _submit_task; grader/durable_jobs.py: submit

HOW THE GRADER WORKS

What stays fixed?

Pinned inputs make grading consistent across tasks and coordinator restarts.

Commit pinning

A snapshot fixes an actual commit, such as A. CPU and GPU use that commit even if the student pushes B while CPU is running. Compatible unfinished snapshots survive coordinator restarts and take priority over a newer HEAD. A new coordinator invocation can discover the newer work once the previous snapshot is finished.

The fingerprint is part of the input

performance_hash() includes the task name, test-script path and contents, SLURM configuration, reduction, submission timeout, and optional setup-script path and contents. A new fingerprint means the same repository commit can need a new evaluation. skip and blocking are not directly hashed; skipping changes which tasks belong to a snapshot.

Change	Expected behavior
New student commit B	Grade B in a later snapshot; keep A's result until replacement.
CPU script changes F1 to F2, no push	Full reconciliation selects repositories; CPU gets a fresh run at the selected commit.
Add a GPU task	Select all submissions on the changed task set; existing CPU evidence may be reused.
Another notification resolves to A/F1 again	Reuse completed evidence; no fresh compute is required.
Change config/repository during unfinished work	Preserve outstanding scheduler holds; supersede incompatible snapshots when safe, abandoning/cancelling active predecessor evaluations as needed.

Two different meanings of "run"
A coordinator invocation is one execution of main.py. An evaluation run is one persisted run_id for a student/task. One evaluation may survive many coordinator invocations and several scheduler attempts.

Successful inputs are fixed within a coordinator invocation. The code does not continuously refetch newly arriving pushes during its waiting loop. Restart or the next scheduled invocation performs fresh discovery.

HOW THE GRADER WORKS

The shared local state

GradingState is the coordinator's durable memory; jobs communicate through result files.

State collection	What it remembers
records	Latest selected result per assignment/student/task, including repository, commit and fingerprint.
runs	Evaluation IDs, fixed inputs, attempt/recovery counters, selected full results and remote publication status.
submission_intents	Tokens, batch membership, scheduler context, known job IDs and submission/recovery state.
jobs	Known in-flight allocations, including their task, students and bundle references.
snapshots	Pinned commit, applicable task fingerprints, evaluation/evidence map and acknowledgement progress.
outbox	Saved immutable finish/cancel/acknowledgement requests waiting for remote confirmation.
attempts / discovery	Compatibility attempt counters, task fingerprints and last successful full-scan metadata.

Files under the shared working directory

```
<working_dir>/
  grading_state.json       # records + runs + intents + outbox
  grader.lock              # exclusive coordinator lock
  repository_bundles/      # pinned local Git bundles
  attempts/<token>/<submission>/  # isolated compute clones
  results/<assignment>/<task>/<token>/<submission>.json
  slurm_logs/              # submitit files and local receipts
```

UUIDs in paths are normalized to the hyphenated UUID spelling; arbitrary identity strings are hashed. The job-name token uses its 32-character hex form. These spellings can differ while representing the same token.

One writer to the coordinator state
Only the coordinator mutates GradingState. Compute jobs write result files. State saves use a temporary file, fsync and atomic rename. The process holds an exclusive lock for the entire coordinator invocation and loads state only after obtaining it.

Version 1 state loads and upgrades to version 2 on save. grades_file is derived output, never the source used to decide what was graded.

HOW THE GRADER WORKS

One pass versus one invocation

A pass is a reconciliation cycle. An invocation may execute one pass or many.

Order	Action	Purpose
1	DurableJobs.reconcile()	Recover/refresh new scheduler intents; release finished work or permit bounded recovery.
2	Drop finished legacy jobs	Compatibility handling for old jobs without tokens.
3	Ingest result files	Validate, save result and outbox, then delete files.
4	Flush Worker publication	Retry saved publication before considering new work.
5	Process assignments	Discover/resume snapshots; submit eligible work within the job limit and blocking policy.
6	Clean unused bundles	Retain those still referenced by jobs or unresolved intents.
7	Flush, finish snapshots, flush	Publish new results, queue ready acknowledgements and try those writes.

wait_for_jobs: true

Repeat passes every 30 seconds while scheduler slots are held or work was deferred by max_submitted_jobs. A held or blocked intent can keep this loop alive. A successful pin stays fixed throughout the invocation.

wait_for_jobs: false

Run one pass and exit. A scheduler or external launcher must invoke the grader again. The grader does not install a schedule or resubmit its own coordinator process. Existing jobs can continue writing shared result files after it exits.

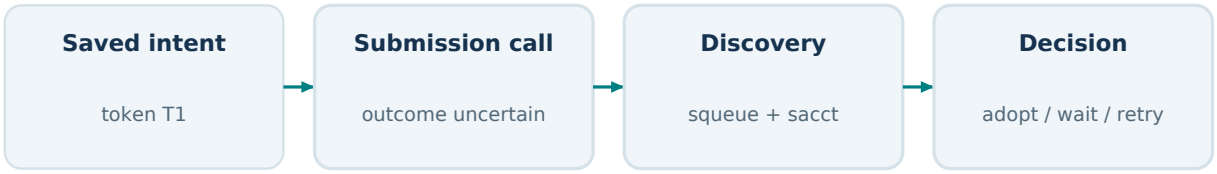
Publication failure alone does not keep the process running
A completed evaluation can have a pending finish or acknowledgement when the invocation exits. A later invocation retries the saved request. Refused/download-failed work also does not by itself keep an otherwise idle waiting invocation alive.

After the final pass, _publish() exports the local leaderboard and optionally pushes it to GitHub. Worker publication happens inside passes, independently of this final export. Multiple flush points do not send the same lifecycle operation repeatedly within one pass; operations are deduplicated per pass.

HOW THE GRADER WORKS

How reconcile() works

It refreshes scheduler facts and local holds. It does not execute grading or publish results.



A unique name lets us find an accepted job. SLURM does not enforce name uniqueness.

Saved state	Meaning	Reconcile action
prepared	Intent saved, call not yet permitted.	Mark rejected so later submission logic can rebuild safely.
submitting / uncertain	Call may have been accepted, or known job became unknown.	Search by exact token in live queue and accounting.
accepted	Job ID returned or discovered.	Refresh job state; adopt terminal or live observations.
finished / rejected	Terminal tracked allocation or definite rejection.	No further discovery for that intent.
presumed_absent	Sustained successful empty lookup allowed recovery.	No scheduling hold, but continue checking for a late original.
blocked	Recovery cap or conflicting allocations.	Hold capacity; continue discovery, with intervention needed if not resolved.

For known job IDs, the runner returns finished states and UNKNOWN; active jobs are omitted from that response. Unknown acceptance triggers token discovery using both squeue and sacct, saved scheduler scope and explicit accounting time bounds.

When a job is adopted, its accepted-attempt count is incremented once for that token. A terminal job is removed from the in-flight job list. A terminal job does not necessarily imply a valid result exists: ingestion and eligibility decisions happen later in the pass.

Reconcile grants permission; it does not resubmit
The function neither reads result files nor submits replacement jobs. Result ingestion runs afterward, before submission decisions. A valid available result therefore normally prevents missing work from being submitted again.

HOW THE GRADER WORKS

Crash recovery, with a clock

Unique names enable lookup. A grace period handles sustained absence.

Case A: accepted job, lost coordinator response

The coordinator saves token T1, SLURM accepts J1, then the coordinator crashes before saving J1. On restart, reconcile searches for gw-T1, finds J1 and adopts it. It does not need a fresh evaluation run or a replacement job.

Case B: submission outcome stays invisible

Time / observation	Local consequence
10:00: submission outcome becomes uncertain	Keep a scheduling hold. Do not immediately duplicate the work.
10:02: queue and accounting both succeed, no match	Start absence_since at 10:02.
10:07: both succeed, still empty	Only 5 minutes elapsed; keep waiting.
10:09: one query fails	Clear the absence clock. Failure is not evidence of absence.
10:12: both succeed, empty	Start a new absence period.
10:22 or later: another successful empty observation	With 600-second grace elapsed, count a presumed-absent event and allow recovery if budget remains.

A replacement gets a new token T2 but keeps the missing evaluation's run ID. If T1 later appears, it is tracked again. Different tokens isolate their files. The first valid result selected during sorted-path ingestion wins; that is not necessarily the attempt that finished earliest.

The two limits count different things
Three confirmed accepted attempts without a usable result lead to a final error grade. max_submission_recoveries: 3 blocks at the **third presumed-absent event**: in an all-uncertain sequence, the initial submission plus two replacements are allowed before blocking.

Grace is measured from successful empty observations, not from submission or crash time. With one pass every ten minutes, recovery can occur much later than ten minutes after a crash. Multiple allocations for the same token remain an operator-reconciliation case.

HOW THE GRADER WORKS

The publication outbox

A failed API request must not erase a result or trigger more compute.



Delete the result file only after the local save. Retry publication without running compute again.

Save first, send afterward

When ingesting a valid result, the coordinator stores the full local result and its immutable completion payload in the same state file before deleting the result file. A lost response is retried with the same run ID and payload. HTTP retries receive fresh authentication signatures.

Failure / response	Behavior
Start cannot be confirmed	Keep the start intent; do not submit the affected batch. Retry on a later pass/invocation.
Finish fails transiently	Keep the local grade and outbox. Retry publication; do not regrade merely to republish.
Acknowledgement says active_runs	Keep its outbox entry; results already published remain visible.
Invalid/conflicting request: e.g. 400, 409, 413	Persist a blocked publication for reconciliation/intervention instead of blind endless retries.
Submission deleted: 404	Mark remote work obsolete and retire relevant pending writes where applicable.

What is sent as the task result?

The payload contains commit_hash, status, error, stdout, runtimes, avg_runtime, data and reduction. Status preserves graded, error or timeout. Empty runtimes produce average 0; this is not proof of a successful measurement.

The Worker body limit is 1 MiB. Only stdout may be shortened, using actual serialized UTF-8 body size and explicit truncation metadata. Full output remains local. If other fields still exceed the limit, publication stays blocked; data is not silently truncated.

Remote cancellation is not scancel
Cancelling a Worker run clears its publication activity. It does not kill a SLURM job. Scheduler holds and live allocations must be dealt with separately before replacement evaluation work is safe.

HOW THE GRADER WORKS

Acknowledging a push

It consumes a matching pending notification, not a compute job or a grade.

Once every applicable task has confirmed completed publication, the coordinator saves an evidence map from task names to completed run IDs. It may mix freshly computed results with previously published results whose repository, commit and fingerprint still match.

Error and timeout results also count as completed evaluations. They are saved and can satisfy a snapshot. A graded status means a usable grading result was produced; data.passed and data.total describe test correctness. Automatic job retries concern missing usable results, not every unsuccessful test.

Situation	Evidence / outcome
CPU and GPU newly completed	Acknowledge update 3 with both run IDs.
Another notification still resolves to the same A/F1/G1	Reuse existing completed run IDs. No extra grading is needed just to consume a notification.
A result was published with null update provenance	It can still be reused as completed evidence for a later known push snapshot.
No applicable tasks	Persist the empty task decision and send an empty evidence map. No Git lookup or compute is required.
Local legacy result has no Worker run ID	Publish matching recorded inputs under a new run without compute; wait for confirmation before using it as evidence.

A new push while old work is running

Alex's update 3 is pinned at commit A. Push 4 arrives while CPU/GPU evaluate A. The grader finishes and publishes A's results under their existing runs. Acknowledging 3 must not delete 4. The matching API returns superseded, and update 4 is discovered independently by a later coordinator invocation.

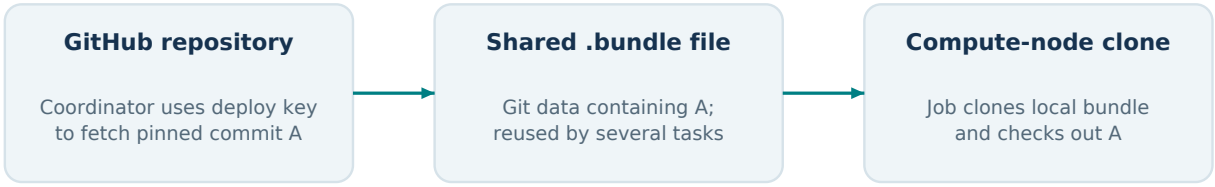
Acknowledgement reason	Outbox action
acknowledged	Matching marker deleted; retire entry.
already_acknowledged	Nothing more to consume; retire entry.
superseded	Newer push version exists; retire only this older intent.
active_runs	Relevant remote work remains active; retain and retry later.

A snapshot can be locally marked finished once its evidence is published and acknowledgement is durably queued. That does not mean the acknowledgement HTTP request has already succeeded. A snapshot with null update_id never queues this push acknowledgement.

HOW THE GRADER WORKS

Inside a compute job

The coordinator downloads Git data; compute nodes grade local copies.



Only the coordinator needs GitHub access. New grading jobs receive no deploy key.

A bundle is Git data in one file

RepositoryStaging downloads the pinned repository using its deploy key, ensures the chosen commit is present, then creates an atomic .bundle file keyed by repository URL and commit. Different tasks can share it. The file contains Git history/files needed for the pinned clone; it is not the student's compiled binary or grading result.

One task job processes its members

• Only rank 0 performs the grading loop. Each student is cloned from the local bundle and explicitly checked out at the pinned commit.

• The trusted grading script is copied into the clone. A student-committed symlink at that destination is removed before copying.

• If configured, a trusted setup script is sourced by a login Bash before the grading script executes. Setup and grading share the timeout.

• The grading script runs in its own process group. Timeout handling terminates the group. The last stdout line is parsed as JSON with passed, total and times.

• Write an atomic result carrying submission ID, run ID, token, commit and fingerprint. Remove this attempt's clone unless preservation is enabled.

Bundle retention is reference-based
Keep a bundle while a known job or unresolved intent might need it. Presumed-absent attempts keep their bundles because an original job could appear late; retention may therefore be indefinite. LFS objects and submodule repositories are not downloaded automatically.

New job arguments contain no deploy keys, Worker admin key or GitHub PAT. This does not constitute a complete filesystem sandbox: student execution isolation and the forgeable last-stdout-line contract remain separate, unresolved concerns.

Source: grader/repository_staging.py; grader/submission_grader.py: grade_all, _run_test_script

HOW THE GRADER WORKS

The modes side by side

The grading and publication rules are the same; discovery chooses the candidates differently.

Start with Alex at commit A, CPU fingerprint F1, already graded and published. Assume no unfinished local work.

Event	Submissions discovery	Updates discovery
No changes	Select Alex; resolve HEAD A; skip compute because saved inputs match.	Normally skip Alex's HEAD check until a full scan is due.
Alex pushes B, webhook pending	Select Alex; resolve B; create new CPU run and publish B.	Select via pending update; resolve B; create new CPU run, publish and acknowledge its snapshot.
Alex pushes B, webhook is missed	Next fresh invocation checks HEAD and notices B.	Periodic full scan notices B even without a notification.
CPU script changes to F2, no push	Select Alex; same commit needs a new CPU evaluation.	Fingerprint change forces full scan; same commit needs a new CPU evaluation.
Add GPU, no push	Select Alex; CPU can be reused; evaluate GPU.	Task-set change forces full scan; CPU can be reused; evaluate GPU.
New notification, still A/F1	Saved result still matches; no compute.	Select Alex, verify A/F1, acknowledge using existing published evidence.

Worked submissions example: no pending push

In a separate initial-grading scenario, Alex has commit A and no local grade. With publication enabled and submissions discovery: **(1)** read S-A from /submissions; **(2)** resolve HEAD A and save a CPU snapshot, using update_id null if no version was supplied; **(3)** create R-A-CPU and confirm the Worker start with that null provenance; **(4)** submit token T1/job J1; **(5)** ingest and publish the CPU result. No pending push is required at any step, and no push acknowledgement is sent for this null-version snapshot.

If a later updates-mode invocation sees update 7 resolving to the same A/F1, it can cite this completed R-A-CPU to acknowledge 7 without executing CPU again.

What if publication is disabled?

Submissions mode continues to save local results, export grades_file and optionally push it to GitHub. Scheduler recovery, pinning and isolated attempts still operate. Worker start/finish/acknowledgement calls are skipped. Updates discovery is not a valid configuration when publication is disabled.

Neither mode blindly trusts a webhook commit
The pending notification selects work and supplies provenance. Actual Git lookup and the persisted snapshot fix the graded commit. In either mode, a compatible unfinished snapshot resumes its saved inputs before moving to newer work.

HOW THE GRADER WORKS

Configuration reference

These controls answer different operational questions. Examples are grader-section fragments.

Field	Default	Meaning
worker_publication	false	Enable the matching run-based Worker lifecycle API.
worker_discovery	submissions	Select repositories via all submissions or pending updates plus reconciliation.
worker_reconcile_seconds	3600	Full-scan interval in updates mode; checked on fresh discovery.
job_mode	per_task	Batch eligible students by task, or one job per student/task.
wait_for_jobs	true	Repeat passes while scheduler holds/deferrals remain; false performs one pass.
max_submitted_jobs	-1	Tracked/reserved scheduler limit; -1 is unlimited.
submission_grace_seconds	600	Successful empty-discovery period before presumed absence.
max_submission_recoveries	3	Block at the third presumed-absent event for an active evaluation.

Publication enabled, scheduled updates discovery

```
grader:
  # Also provide working_dir, grades_file, worker_url,
  # worker_private_key and the assignment/task configuration.
  worker_publication: true
  worker_discovery: "updates"
  worker_reconcile_seconds: 3600
  job_mode: "per_task"
  wait_for_jobs: false
  max_submitted_jobs: 4
  submission_grace_seconds: 600
  max_submission_recoveries: 3
```

For submissions discovery, change worker_discovery to "submissions"; leave publication true if the Worker should serve results. The independent pusher configuration controls GitHub publication. These examples do not install a scheduler for the coordinator.

The job limit is not a proof of physical cluster occupancy
Uncertain intents reserve slots. Presumed-absent intents release them, even though an original job could appear later. Thus recovery can temporarily create more actual allocations than the tracked limit. CPU blocking also holds back later tasks within its assignment.

HOW THE GRADER WORKS

When something appears stuck

Look at compute state, local result state and remote publication separately.

Symptom	Likely explanation / next check
Worker says grading, no running job	The remote start may precede allocation start; or a scheduler intent is uncertain/blocked. Check the saved token and scheduler scope.
Job finished but public result is old	The result may await ingestion, or its finish outbox request may be pending/blocked. A later coordinator invocation retries transient publication.
Update still pending after CPU grade	Other applicable tasks may be unfinished, publication unconfirmed, acknowledgement blocked, or a newer push may be pending.
No HEAD checks in updates mode	May be expected: no pending notification, unfinished snapshot or due full scan. First/task-change scans behave differently.
Grace period never expires	Failed/invalid scheduler observations reset it. With sparse invocations, successful observations can be far apart.
Recovery stops after several missing submissions	The presumed-absence budget is exhausted. blocked intents hold capacity for reconciliation.
Bundle directory keeps growing	Presumed-absent intents retain their source bundles until terminal discovery or deliberate resolution.
Local fallback cannot recover a missing submission	Local receipts support ID recovery and PID-start checks; a missing receipt does not support SLURM-style absence recovery.

Current operational boundaries

There is no new operator recovery CLI. Resolve scheduler uncertainty against logs, saved intents and accounting before changing state or removing bundles. Repeated failures and conflicting allocations can still require intervention. Remote cancellation does not stop compute.

The matching Worker API must be deployed separately. The new coordinator paths have local regression coverage; exact name lookup, accounting retention, shared paths and scheduler behavior still need verification on Leonardo. Historical token-less jobs retain their legacy compatibility handling.

Output is public when published
Worker and optional GitHub output can include stdout/error details. Deploy keys in discovery responses are secret. New compute arguments omit credentials, but full code-execution sandboxing is not implemented by these recovery changes.

HOW THE GRADER WORKS

Code map and quick answers

Use this page to connect the explanation back to the implementation.

Read this file / function	To understand
grader/grader.py _snapshot, _discover_updates	Discovery selection, cached assignment lists, commit pinning and full reconciliation.
grader/grader.py _submit_assignment, _submit_task	Eligibility, blocking tasks, job limit, staging and batch submission.
grader/durable_jobs.py ensure_run, submit, reconcile, _adopt	Evaluation identities, durable scheduler intents, job recovery and accepted-attempt accounting.
grader/grading_state.py save, record, needs_grading	State schema, atomic persistence, latest results and input comparisons.
grader/worker_publication.py start, enqueue, flush, enqueue_ack	Remote run lifecycle, immutable outbox and acknowledgement dispositions.
runners/SlurmRunner.py submit, finished_jobs, discover_submission	submitit parameters, SLURM name lookup, context validation and local receipts.
grader/repository_staging.py stage, cleanup	Pinned bundles, credential use in the coordinator and bundle retention.
grader/submission_grader.py grade_all, _run_test_script	Compute execution, isolated attempt files, setup and timeout behavior.
grader/publication.py; config/configs.py	Payload validation/truncation, configuration defaults and task fingerprints.

Four quick answers

Does GET /updates remove an update? No. A separate acknowledgement consumes the matching pending version.

Is an active Worker run proof that code is executing? No. Remote activity begins before scheduling and can span retries or publication delay.

Does a publication retry need a new SLURM job? No. It reuses the saved completion payload and run ID.

What does DurableJobs interact with? The shared GradingState and scheduler runner. WorkerPublication handles the Worker API.

The Worker-facing behavior described here follows the run-based API contract supplied in the conversation and used by the client. Remote implementation and deployment were not inspected or changed for this document. All examples use fictitious student labels and redacted credentials.